Software Engineer — Step 3

Databases & SQL

Roadmap objective: Understand persistent data, relational modeling and querying
Level: Beginner → Intermediate
Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Relational model
- Keys
- Normalization
- CRUD
- Joins
- Indexes
- Transactions
- Constraints
- Query planning basics

Core concepts

1. Software engineers should understand the data model behind their applications.

2. Normalization reduces redundancy but excessive normalization can make reads more complex; model around actual requirements.

3. Indexes can make reads faster but consume space and can slow writes.

4. Transactions are essential when multiple changes must preserve a consistent business state.

Concept map

```
Databases & SQL
  |
  +--> Learn the concept
  |
  +--> Reproduce a small example
  |
  +--> Apply it in a feature
  |
  +--> Test failure/edge cases
  |
  +--> Document the decision
```

Detailed study guide

1. Relational model

Study relational model through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Keys

Study keys through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Normalization

Study normalization through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. CRUD

Study crud through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Joins

Study joins through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Indexes

Study indexes through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Transaction concept

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 500
WHERE id = 1;

UPDATE accounts
SET balance = balance + 500
WHERE id = 2;

COMMIT;

-- Use ROLLBACK if a required operation fails.
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Normalization	Reduce redundancy	Relational design
Index	Speed selected access	Storage/write cost
Transaction	Atomic unit of work	Consistency

Common mistakes

- No foreign keys where relationships matter
- Indexes everywhere
- Long transactions
- N+1 queries
- Ignoring transaction failure paths

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create three related tables.
- Write down what you expected, what happened, and why.

Lab 2

- Insert realistic sample data.
- Write down what you expected, what happened, and why.

Lab 3

- Write joins and aggregation queries.
- Write down what you expected, what happened, and why.

Lab 4

- Add constraints.
- Write down what you expected, what happened, and why.

Lab 5

- Create one useful index.
- Write down what you expected, what happened, and why.

Lab 6

- Demonstrate COMMIT and ROLLBACK.
- Write down what you expected, what happened, and why.

Mini-project

Design a transactional banking or order database. Demonstrate keys, constraints, joins, indexes and a transaction that either completes fully or rolls back.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is normalization?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why use transactions?

- Answer structure: definition → example → trade-off/limitation → practical use.

What does an index do?

- Answer structure: definition → example → trade-off/limitation → practical use.

INNER JOIN vs LEFT JOIN?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a foreign key?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you find a slow query?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Relation	Write this in your own words.	Give one project example.
Normalization	Write this in your own words.	Give one project example.
Constraint	Write this in your own words.	Give one project example.
Index	Write this in your own words.	Give one project example.
Transaction	Write this in your own words.	Give one project example.
Isolation	Write this in your own words.	Give one project example.
Join	Write this in your own words.	Give one project example.
Query plan	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Foreign key	Write this in your own words.	Give one project example.
Atomicity	Write this in your own words.	Give one project example.

Self-test

- Explain Relational model without using its textbook definition.
- Show a small example of Keys.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Relational model.
- I can explain and demonstrate Keys.
- I can explain and demonstrate Normalization.
- I can explain and demonstrate CRUD.
- I can explain and demonstrate Joins.
- I can explain and demonstrate Indexes.
- I can explain and demonstrate Transactions.
- I can explain and demonstrate Constraints.
- I can explain and demonstrate Query planning basics.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- https://dev.mysql.com/doc/

Recommended videos

- CodeWithHarry — SQL Tutorial for Beginners (Complete Course using MySQL)
  https://www.youtube.com/watch?v=yE6tIle64tU (NEW)
- MongoDB Tutorial (Existing DB resource 103)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.